МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

“Санкт-Петербургский национальный исследовательский университет информационных технологий механики и оптики”

Мегафакультет: трансляционных информационных технологий

Факультет: информационных технологий и программирования

Лабораторная работа №1

По дисциплине: “Интеллектуальные системы и технологии”

Тема: “Классификация эмоционального состояния человека по сигналам электроэнцефалографии”

Выполнил студент группы №M3415:

Ким Александр Николаевич

НИУ ИТМО, САНКТ ПЕРЕРБУРГ

2026

Цель работы
Изучить метод решения задачи распознавания эмоций по сигналам электроэнцефалографии (ЭЭГ) и построить алгоритм классификации эмоционального состояния человека по многоканальной записи ЭЭГ в трёхклассовой постановке: положительная эмоция (+1), нейтральное состояние (0), отрицательная эмоция (-1).

Описание данных
В работе используется открытый исследовательский набор SEED. Датасет содержит записи ЭЭГ 15 испытуемых, каждый из которых прошёл 3 сессии с интервалом не менее недели. В каждой сессии предъявлялось 15 видеоклипов длительностью около 4 минут, вызывающих целевое эмоциональное состояние. Регистрация проводилась системой ESI NeuroScan с 62 электродов по международной системе 10-20. Исходная частота дискретизации 1000 Гц была понижена авторами датасета до 200 Гц, выполнена полосовая фильтрация 0-75 Гц и удаление фрагментов с грубыми артефактами.

В используемом подмножестве доступно 44 файла сессий, что составляет 660 проб. Метки классов одинаковы для всех сессий и сбалансированы: по 220 проб на каждый из трёх классов. Каждая проба представляет собой матрицу размером 62 канала на T отсчётов.

Загрузка меток и списка файлов сессий выполняется так:

```
labels = sio.loadmat(os.path.join(DATA_DIR, 'label.mat'))['label'].flatten()
session_files = list_session_files(DATA_DIR)

print('Метки 15 проб сессии:', labels)
print()
print('Всего файлов сессий:', len(session_files))
print('Первые:', session_files[:5])
print('Последние:', session_files[-3:])

n_subjects = len({f.split('_')[0] for f in session_files})
n_sessions = len(session_files)
print()
print(f'Испытуемых: {n_subjects}  -  сессий в наборе: {n_sessions}')
print(f'Всего проб: {n_sessions} x 15 = {n_sessions * 15}')

y = np.tile(labels, n_sessions)  # метки одинаковы для каждой сессии
print('Баланс классов (все пробы):',
      dict(zip(CLASS_NAMES, np.bincount(y + 1))))

[2]
```

```
Метки 15 проб сессии: [ 1  0 -1 -1  0  1 -1  0  1  1  0 -1  0  1 -1]

Всего файлов сессий: 44
Первые: ['10_20131130.mat', '10_20131204.mat', '10_20131211.mat', '11_20140618.mat',
'11_20140625.mat']
Последние: ['9_20140620.mat', '9_20140627.mat', '9_20140704.mat']

Испытуемых: 15  -  сессий в наборе: 44
Всего проб: 44 x 15 = 660
Баланс классов (все пробы): {'negative': np.int64(220), 'neutral': np.int64(220),
'positive': np.int64(220)}
```

Предобратока

Несмотря на предварительную подготовку авторами датасета, сигнал требует дополнительной обработки перед подачей в классификатор. Для каждой пробы выполняется следующий пайплайн:

1. Удаление тренда. Из каждого канала вычитается его среднее значение.
2. Режекторный фильтр 50 Гц. Применяется фильтр iirnotch с добротностью Q=30 для подавления сетевой наводки.
3. Полосовой фильтр 0.5-45 Гц. Фильтр Баттерворта 4-го порядка оставляет только информативные для ЭЭГ частотные полосы.
4. Z-нормализация. Каждый канал приводится к нулевому среднему и единичной дисперсии.

```python
def preprocess_trial(trial, fs=200):
    trial = trial - np.mean(trial, axis=1, keepdims=True) # ремуваем тренд
    b, a = iirnotch(50 / (fs / 2), Q=30) # удаляем частоты
    trial = filtfilt(b, a, trial, axis=1)
    nyq = fs / 2
    b, a = butter( N: 4,  Wn: [0.5 / nyq, 45 / nyq], btype='band') # по нормировке
    return filtfilt(b, a, trial, axis=1)


def normalize_trial(trial): # Z ахахах avg = 0 и D = 1
    mean = np.mean(trial, axis=1, keepdims=True)
    std = np.std(trial, axis=1, keepdims=True)
    std[std == 0] = 1 # не делить на 0, если дисперсия канала = 0
    return (trial - mean) / std


def process_one_file(file_path, n_trials=15):
    trials = load_trials_from_file(file_path)
    X = np.empty((n_trials, 62 * FEATS_PER_CHANNEL))
    for i, trial in enumerate(trials):
        trial = preprocess_trial(trial)
        trial = normalize_trial(trial)
        X[i] = extract_features_from_trial(trial)
    return X
```

Потом для каждой пробы по каждому из 62 каналов вычисляется 14 признаков, формирующих вектор длиной 868 (62 на 14):

- Статистические признаки (8): среднее, стандартное отклонение, дисперсия, асимметрия, эксцесс, минимум, максимум, разница между макс и мин.
- Спектральные признаки (5): относительная мощность в пяти классических полосах ЭЭГ: дельта, тета, альфа, бета, гамма.
- Признак сложности (1).

Сложность основана на понятии Mobility (Подвижность).

- Подвижность сигнала X— это корень из отношения дисперсии производной сигнала к дисперсии самого сигнала. Грубо говоря, это средняя частота колебаний.
- Complexity (Сложность) — это отношение подвижности производной сигнала к подвижности *самого* сигнала.

$$\text{Complexity} = \frac{\text{Mobility}(dX)}{\text{Mobility}(X)}$$

Расчёт признаков распараллелен по файлам сессий с использованием пула потоков и кэшируется

```
paths = [os.path.join(DATA_DIR, f) for f in session_files]
t_start = time.time()

X = None
if os.path.exists(CACHE_FILE):
    cached = np.load(CACHE_FILE)
    if cached.shape[0] == len(paths) * 15:   # кэш по числу проб
        X = cached
        print(f'Признаки загружены из кэша: {X.shape}')

if X is None:
    print(f'Вычисляем признаки для {len(paths)} файлов '
          f'в {N_WORKERS} потоках (может занять около 2-3 мин)...')
    with ThreadPoolExecutor(max_workers=N_WORKERS) as pool:
        blocks = list(pool.map(process_one_file, paths))
    X = np.concatenate(blocks, axis=0)
    np.save(CACHE_FILE, X)
    print(f'Признаки вычислены и сохранены в кэш: {X.shape}')

print(f'Затрачено: {time.time() - t_start:.1f} с')
print('Конечность значений:', np.isfinite(X).all())
print('Матрица X:', X.shape, '| вектор y:', y.shape)

[5]
```

```
Признаки загружены из кэша: (660, 868)
Затрачено: 0.0 с
Конечность значений: True
Матрица X: (660, 868) | вектор y: (660,)
```

Протокол разбиения и масштабирования

Данные разделяются на обучающую (70%, 462 пробы) и тестовую (30%, 198 проб) выборки с применением стратификации. Масштабирование выполняется с помощью StandardScaler, параметры которого вычисляются исключительно на обучающей выборке.

```python
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=SEED, stratify=y)

scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train)
X_test_std = scaler.transform(X_test)

print(f'Обучающая выборка: {X_train_std.shape[0]} проб')
print(f'Тестовая выборка:  {X_test_std.shape[0]} проб')
print('Классы в train:', dict(zip(CLASS_NAMES, np.bincount(y_train + 1))))
```

Модели классификации

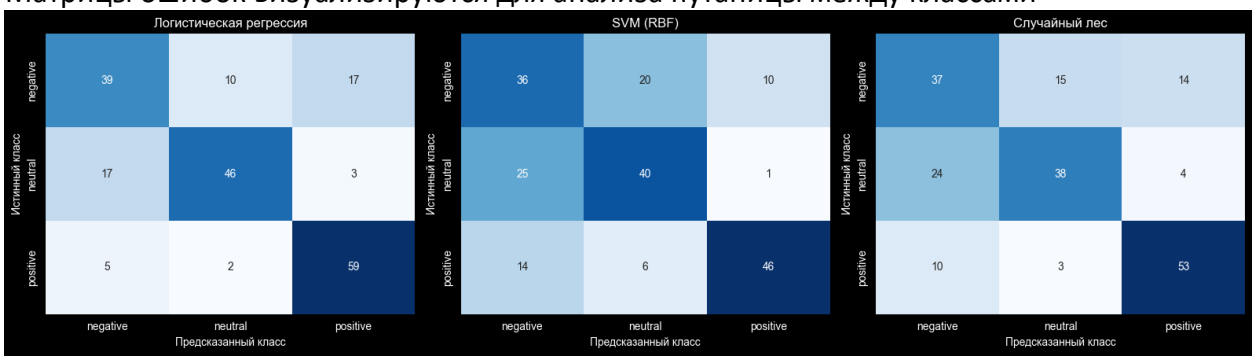
Для решения задачи классификации были реализованы три алгоритма: логистическая регрессия, SVM с RBF-ядром и случайный лес.

```python
classifiers = {
    'Логистическая регрессия': LogisticRegression(max_iter=1000, random_state=SEED),
    'SVM (RBF)': SVC(kernel='rbf', random_state=SEED, class_weight='balanced'),
    'Случайный лес': RandomForestClassifier(n_estimators=100, random_state=SEED),
}
```

3 rows × 4 cols

	accuracy	precision	recall	f1
Логистическая регрессия	0.7273	0.7264	0.7273	0.7233
SVM (RBF)	0.6162	0.6310	0.6162	0.6216
Случайный лес	0.6465	0.6487	0.6465	0.6456

Матрицы ошибок визуализируются для анализа путаницы между классами



Для оценки устойчивости моделей проведена стратифицированная 3-кратная кросс-валидация на обучающей выборке

Вместо того чтобы один раз разделить данные на обучение и тест, мы делаем это 3 раза по особому правилу:

1. Вся обучающая выборка (462 пробы) делится на 3 равные части - фолды.
2. Итерация 1: Модель обучается на частях 1 и 2, а проверяется на части 3.
3. Итерация 2: Модель обучается на частях 1 и 3, а проверяется на части 2.
4. Итерация 3: Модель обучается на частях 2 и 3, а проверяется на части 1.

Стратификация лишь гарантирует, что в каждой из 3 частей сохраняется исходная пропорция классов.

Так мы получаем среднюю точность по трем прогонам +- устойчивость

```
cv = StratifiedKFold(n_splits=3, shuffle=True, random_state=SEED)
cv_scores = {}
print('Стратифицированная 3-кратная кросс-валидация на обучающей выборке:')
for name, clf in classifiers.items():
    scores = cross_val_score(clf, X_train_std, y_train, cv=cv, scoring='accuracy')
    cv_scores[name] = scores
    print(f'  {name:24s} Accuracy: {scores.mean():.4f} плюс минус {scores.std():.4f} '
          f'  части: {np.round(scores, 4)}')
```

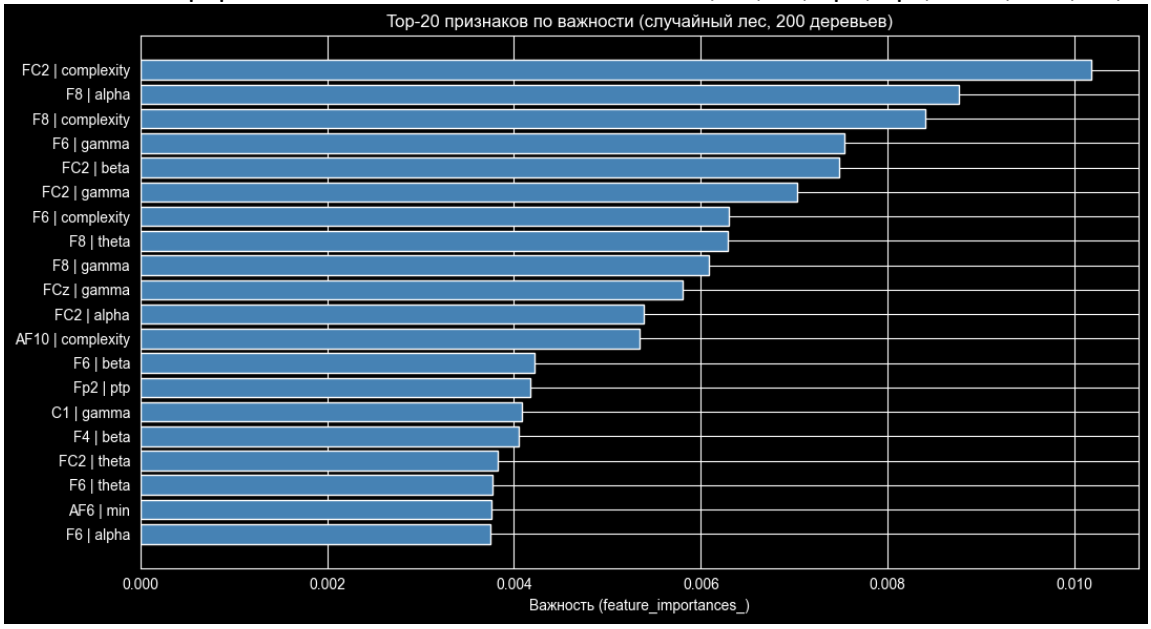
Анализ важности признаков выполнен на основе случайного леса из 200 деревьев. Важность канала определяется как средняя важность его 14 признаков.

```
rf_imp = RandomForestClassifier(n_estimators=200, random_state=SEED)
rf_imp.fit(X_train_std, y_train)
importance = rf_imp.feature_importances_

feature_labels = [f'{CH_NAMES[c]} | {FEATURE_NAMES[f]}'
                  for c in range(len(CH_NAMES))
                  for f in range(FEATS_PER_CHANNEL)]

top_idx = np.argsort(importance)[::-1][:20]
```

Наиболее информативными оказались каналы: FC2, F8, F6, Fp2, Fpz, AF10, AF9, F4, C1, TP7.



Проверена гипотеза о достаточности признаков только 10 наиболее важных каналов (140 признаков вместо 868).

```python
selected = np.concatenate([
    np.arange(c * FEATS_PER_CHANNEL, c * FEATS_PER_CHANNEL + FEATS_PER_CHANNEL)
    for c in top_channels])
print('Используем каналы:', ', '.join(CH_NAMES[c] for c in top_channels),
      '| признаки:', len(selected))

X_train_top = X_train_std[:, selected]
X_test_top = X_test_std[:, selected]

reduced_results = {name: evaluate(clone(clf), X_train_top, X_test_top,
                                  y_train, y_test)
                   for name, clf in classifiers.items()}

pd.DataFrame({name: {k: round(v[k], 4) for k in
                     ('accuracy', 'precision', 'recall', 'f1')}
              for name, v in reduced_results.items()}).T
```

	accuracy	precision	recall	f1
Логистическая регрессия	0.6667	0.6607	0.6667	0.6604
SVM (RBF)	0.6566	0.6631	0.6566	0.6559
Случайный лес	0.7071	0.7096	0.7071	0.7070

Выводы

В ходе лабораторной работы разработан и исследован полный конвейер классификации эмоциональных состояний по данным ЭЭГ датасета SEED.

1. Логистическая регрессия показала худшее качество классификации (Accuracy 0.66 на сокращённом наборе) и наибольшую устойчивость на кросс-валидации.
2. Положительная эмоция распознаётся наиболее точно, тогда как классы negative и neutral демонстрируют наибольшую взаимную путаницу.
3. Информативными являются лобные и фронтально-центральные зоны коры, а ключевыми признаками: спектральные мощности в гамма-, альфа- и тета-полосах и сложность сигнала.
4. Сокращение набора признаков до 140 улучшает качество классификации по сравнению с полным набором из 868 признаков.